

PROBLEMAS SEMANA 8

Archivos Binarios, struct y Registros de Longitud Fija, y Registros de Longitud Variable

Introducción

Los problemas de esta semana introducen el trabajo con **archivos binarios** como medio para la persistencia eficiente de grandes colecciones de datos homogéneos. El eje temático es la **representación directa de datos en bytes** mediante el módulo struct y el aprovechamiento del **acceso directo por offset** que permite una sola llamada al sistema operativo (seek) para llegar al k-ésimo registro sin recorrer los anteriores. Sobre esa base, se construye el patrón **ABM** (Alta, Baja lógica, Modificación) con gestión de espacio libre por **free list encadenada**, y se verifica empíricamente la diferencia de rendimiento entre acceso secuencial y acceso directo.

Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa modular** consolidado desde la Semana 4: cada función incluye su sección declarativa (docstring con precondition, postcondición y, cuando corresponda, **efectos secundarios** sobre el archivo en disco). Las constantes globales (formato, tamaño del registro, número mágico) se declaran una sola vez como **fuentes únicas de verdad** y se derivan unas de otras con struct.calcsiz.

Los cinco problemas están organizados en tres bloques con **conexiones deliberadas**: el Bloque A construye los cimientos —empaquetado de datos elementales y diseño del registro—; el Bloque B ataca el patrón ABM completo con free list y verificación visual del archivo binario mediante volcado hexadecimal; y el Bloque C mide empíricamente la ganancia del acceso directo sobre el secuencial y verifica que la representación elegida en struct respeta lo aprendido en la Semana 2 sobre complemento a dos e IEEE 754. Se agrega una sección de **Desafíos extracurriculares grupales** con cuatro propuestas que profundizan los temas conceptualmente introducidos pero no implementados en la teoría (bitmap de espacio libre, slotted pages, log append-only y migración entre formatos de serialización).

Recordatorio: Modelo de programa modular — Cada función incluye su sección declarativa como docstring (descripción, precondition, postcondición, efectos secundarios si los hay), accesible mediante help(). La sección algorítmica se organiza en Prólogo / Resolución / Epílogo. Las constantes (FORMATO, TAM_REGISTRO, MAGIC) se declaran una vez y se derivan automáticamente: TAM_REGISTRO = struct.calcsiz(FORMATO).

Herramientas clave de inspección — Para verificar el contenido real de los archivos binarios producidos en clase, se utilizarán xxd (Linux/macOS) o hexdump (universal en Unix), que muestran el archivo byte a byte en hexadecimal con su columna ASCII a la derecha. En Windows, los equivalentes son Format-Hex (PowerShell) o herramientas como HxD. La inspección hexadecimal es la única forma directa de validar que los bytes producidos por struct.pack coinciden con la representación esperada.

Conexión con la Semana 2 — Los problemas del Bloque C retoman la representación binaria de enteros (complemento a dos para enteros con signo) y de números reales (IEEE 754 para float/double). El módulo struct materializa esa representación en bytes concretos: el experimento del Problema 5 cierra el ciclo abierto en la Semana 2 cuando todavía no teníamos manera de "ver" los bytes de un float.

Bloque A: Fundamentos — struct y diseño del registro

Problema 1: Empaquetado y desempaquetado guiado con struct

Enunciado

Escribir un programa en Python que realice los siguientes experimentos con el módulo struct, registrando las predicciones *antes* de ejecutar cada bloque y verificándolas después con `len()` y con un volcado hexadecimal manual:

Experimento 1 — Tipos elementales:

- (a) Empaquetar el entero 42 con cuatro formatos distintos: 'b' (int8), 'h' (int16), 'i' (int32), 'q' (int64). Predecir antes de ejecutar: ¿cuántos bytes ocupa cada uno? Verificar con `len(struct.pack(...))` y mostrar los bytes con `.hex()`.
- (b) Empaquetar el flotante 3.14 con 'f' (float32) y con 'd' (float64). Comparar tamaños y bytes producidos. ¿Qué diferencia se observa entre la representación de 32 y 64 bits?
- (c) Empaquetar la cadena b'Ada' con tres formatos: '3s', '10s' y '2s'. Predecir y verificar: ¿qué pasa con la cadena cuando el formato es más corto que la cadena? ¿Y cuando es más largo? ¿Con qué bytes se rellena?

Experimento 2 — Endianness:

- (d) Empaquetar el entero 305419896 (que en hexadecimal es 0x12345678) con tres prefijos: '<i' (little-endian), '>i' (big-endian) y '!i' (network byte order). Comparar los bytes producidos. ¿Qué relación hay entre > y !?
- (e) Sin usar el prefijo ('i' solo), empaquetar 305419896 y comparar con los resultados de (d). ¿Qué endianness se usa por defecto en esta máquina? Justificar consultando la documentación del módulo struct.

Experimento 3 — Desempaquetado y validaciones:

- (f) Crear `bytes_originales = struct.pack('<ii', 100, 200)`. Desempaquetar con `struct.unpack('<ii', bytes_originales)`. Verificar que el resultado es la tupla (100, 200). Luego intentar desempaquetar con '<i' (un solo entero, formato más corto que los datos): registrar el error y explicar la causa.
- (g) Calcular `struct.calcsize('<i32s16s40s')` y verificar que coincide con la suma manual ($4 + 32 + 16 + 40 = 92$ bytes). ¿Por qué es importante usar `calcsize` en lugar de hardcodear el número 92 en el programa?

Orientaciones para la resolución

Análisis: Antes de cada experimento, anotar en un comentario la predicción (cuántos bytes, qué bytes esperados, qué endianness). Esto transforma el ejercicio de "ejecutar y ver" en "predecir, ejecutar y comparar".

Diseño: Organizar cada experimento en una función separada que reciba los datos, los empaquete, imprima `len()`, los bytes en hexadecimal con `.hex(' ')` (separador opcional para legibilidad) y los valores desempaketados. La sección algorítmica del programa principal invoca cada función con los casos de prueba.

Evaluación: Comparar la salida con los volcados hexadecimales esperados, calculados manualmente para los casos pequeños. Para `305419896 = 0x12345678`, los bytes en little-endian deben ser `78 56 34 12`; en big-endian, `12 34 56 78`.

*Conexión con la clase teórica — Este problema ejercita directamente las Secciones 3.1 a 3.4 del documento teórico: `pack`, `unpack`, **cadenas de formato**, **endianness** y `calcsize`. La constante derivada con `calcsize` (Sección 3.4) es la base de todos los problemas siguientes: ningún número entero "92" o "97" debe aparecer hardcodedo en el código.*

Problema 2: Diseño del registro de agenda y operaciones básicas

Enunciado

Implementar las operaciones básicas sobre un archivo de agenda de contactos con registros de longitud fija. El registro tiene la estructura especificada en la Sección 4.2 del documento teórico:

Campo	Tipo struct	Tamaño
id	i (int32)	4 bytes
nombre	32s	32 bytes
telefono	16s	16 bytes
email	40s	40 bytes

El formato es `'<i32s16s40s'` y `TAM_REGISTRO = struct.calcsize(FORMATO) = 92`. Implementar las siguientes funciones:

(a) `empaquetar_contacto(id, nombre, telefono, email)` que reciba los datos como Python nativos (int y str) y devuelva los 92 bytes listos para escribir. Las cadenas se codifican en UTF-8 antes de empaquetar y se truncan al máximo si superan el tamaño del campo. **Documentar en la docstring** qué pasa con caracteres no ASCII (1 carácter UTF-8 puede ocupar más de un byte).

(b) `desempaquetar_contacto(bytes_registro)` que reciba 92 bytes y devuelva la tupla (`id`, `nombre`, `telefono`, `email`) con las cadenas decodificadas a `str` y con los bytes nulos de relleno removidos (usar `.rstrip(b'\x00').decode('utf-8')`).

(c) `crear_agenda(ruta, lista_contactos)` que reciba la ruta del archivo a crear y una lista de tuplas (`id`, `nombre`, `telefono`, `email`), abra el archivo en modo `'wb'` y escriba todos los contactos en orden. **Efecto secundario:** crea o sobrescribe el archivo en `ruta`.

(d) `leer_contacto(archivo, k)` que reciba un archivo abierto en modo `'rb'` o `'r+b'` y un índice `k`, posicione el cursor con `seek(k * TAM_REGISTRO)`, lea exactamente `TAM_REGISTRO` bytes y devuelva la tupla desempaquetada. Es $O(1)$ sin importar el tamaño del archivo.

(e) `actualizar_email(ruta, k, nuevo_email)` que actualice el email del registro `k`-ésimo *in situ*, sin reescribir el archivo. Abrir en `'r+b'`, leer el registro, modificar solo el email, reempaquetar y escribir de vuelta en la misma posición. **Efecto secundario:** modifica el archivo en `ruta`.

Probar las cinco funciones con una agenda inicial de tres contactos: Ada Lovelace, Alan Turing y Grace Hopper, y luego actualizar el email de Alan Turing. Verificar leyendo el contacto modificado y los otros dos para confirmar que solo cambió el email del segundo.

Orientaciones para la resolución

Análisis: El registro tiene cuatro campos heterogéneos. La función `empaquetar` debe convertir los `str` de Python a bytes UTF-8 antes de pasar a `struct.pack`. La función `desempaquetar` debe invertir el proceso, removiendo el padding de ceros que `struct` agregó al empaquetar cadenas más cortas que el campo.

Diseño: Declarar `FORMATO = '<i32s16s40s'` y `TAM_REGISTRO = struct.calcsize(FORMATO)` como constantes globales al inicio del programa. Toda función que necesite conocer el tamaño del registro lo obtiene de `TAM_REGISTRO`, nunca como literal. Para el truncado de cadenas largas, usar `nombre.encode('utf-8')[:32]`. Documentar el efecto secundario de `crear_agenda` y `actualizar_email`.

Evaluación: Después de `crear_agenda`, verificar que `os.path.getsize(ruta) == 3 * TAM_REGISTRO`. Después de `actualizar_email`, verificar que `os.path.getsize(ruta)` no cambió (la actualización *in situ* no agrega ni quita bytes).

Conexión con la Semana 6 — En la Semana 6 trabajamos con archivos de texto donde modificar una línea exigía reescribir todo el archivo. Aquí, gracias a la longitud fija, `actualizar_email` modifica exactamente 92 bytes de un archivo de cualquier tamaño en tiempo $O(1)$. Esta diferencia es uno de los principales motivos por los que las bases de datos no usan archivos de texto para sus tablas internas.

Bloque B: ABM completo con free list y verificación hexadecimal

Problema 3: ABM con free list encadenada y volcado hexadecimal

Enunciado

Extender el formato del registro para soportar baja lógica y reutilización de espacio mediante free list encadenada, según lo especificado en la Sección 5 del documento teórico. El formato extendido es '`<Bi i32s16s40s`' (97 bytes), donde se agregan al comienzo los campos `activo` (uint8) y `siguiente_libre` (int32). El archivo lleva una cabecera de 12 bytes con formato '`<4sii`': número mágico `b'AGD1'`, cantidad de registros y cabeza de la free list (-1 si está vacía).

Implementar:

(a) `inicializar_archivo(ruta)` que cree el archivo con la cabecera correcta: número mágico, cantidad = 0, `cabeza_libre` = -1. **Efecto secundario:** crea o sobrescribe el archivo.

(b) `leer_cabecera(archivo)` y `escribir_cabecera(archivo, cantidad, cabeza_libre)` que manipulen los 12 bytes iniciales. La función de escritura siempre hace `seek(0)` antes de escribir.

(c) `alta(ruta, id, nombre, telefono, email)` que dé de alta un contacto. Si la free list tiene cabeza válida (≥ 0), reutilizar el registro en esa posición y actualizar `cabeza_libre` con el valor que tenía el campo `siguiente_libre` del registro reutilizado. Si la free list está vacía, agregar el registro al final del archivo e incrementar cantidad. Devolver el índice (físico) donde se escribió el registro.

(d) `baja(ruta, k)` que marque el registro `k` como borrado: pone `activo` = 0, escribe en `siguiente_libre` el valor actual de `cabeza_libre`, y actualiza `cabeza_libre` = `k`. Verificar antes de operar que el registro `k` esté activo (si no, levantar `ValueError`).

(e) `modificacion(ruta, k, nuevo_email)` que actualice el email del registro `k` *in situ*. Verificar que el registro esté activo.

(f) `listar_activos(ruta)` que recorra el archivo desde el primer registro hasta el último y devuelva la lista de tuplas (`k, id, nombre, telefono, email`) para los registros con `activo` == 1, en orden de índice físico.

(g) Verificación con volcado hexadecimal: después de ejecutar la siguiente secuencia, ejecutar `xxd nombre_archivo.dat` (o equivalente) y analizar el resultado, identificando byte por byte: cabecera, registros activos, registros borrados con sus enlaces de free list.

```
inicializar_archivo('agenda.dat')
alta('agenda.dat', 1, 'Ada Lovelace', '4567-1122', 'ada@fi.uba.ar')
alta('agenda.dat', 2, 'Alan Turing', '4511-8890', 'alan@fi.uba.ar')
alta('agenda.dat', 3, 'Grace Hopper', '4533-2214', 'grace@fi.uba.ar')
alta('agenda.dat', 4, 'Linus Torvalds', '4598-5567', 'linus@fi.uba.ar')
baja('agenda.dat', 1)                # borra Alan Turing
baja('agenda.dat', 3)                # borra Linus Torvalds
```

```

indice_nuevo = alta('agenda.dat', 5, 'Margaret Hamilton', '4577-8801',
'margaret@fi.uba.ar')

print(f"Margaret se escribió en el slot {indice_nuevo}")

```

Predecir antes de ejecutar: ¿en qué índice físico quedará Margaret Hamilton? ¿Por qué? Verificar que `listar_activos` devuelve los tres contactos activos en el orden correcto. Inspeccionar el volcado hexadecimal y confirmar dónde aparece la cadena Margaret.

Orientaciones para la resolución

Análisis: La free list opera como una pila LIFO: la última baja queda primera para ser reutilizada. Después de `baja(1)` y `baja(3)`, la cadena es `cabeza_libre → 3 → 1 → -1`. La próxima alta consume el slot 3 (no el 1), y `cabeza_libre` queda apuntando a 1.

Diseño: Definir las constantes `FORMATO_REG = '<Bi i32s16s40s'`, `TAM_REGISTRO = struct.calcsize(FORMATO_REG) (= 97)`, `FORMATO_CAB = '<4sii'`, `TAM_CABECERA = struct.calcsize(FORMATO_CAB) (= 12)` y `MAGIC = b'AGD1'`. Definir la función `offset_de(k)` que calcula `TAM_CABECERA + k * TAM_REGISTRO` para encapsular la aritmética de posicionamiento. Documentar el efecto secundario de alta, baja y modificación en sus docstrings.

Evaluación: El volcado hexadecimal después de la secuencia debe mostrar: cabecera con cantidad = 4 y `cabeza_libre = 1`, registro 0 activo (Ada), registro 1 borrado con `siguiente_libre = -1`, registro 2 activo (Grace), registro 3 ahora activo con los datos de Margaret. Si la inspección no coincide con esa traza, hay un error en la lógica de la free list.

Conexión con la clase teórica — Este problema implementa íntegramente lo desarrollado en la Sección 5 del documento (free list encadenada, baja lógica, reutilización LIFO). El cuadro "Patrón: el archivo como estructura de datos persistente" anticipa que estamos implementando manualmente lo que en la Semana 10 será una lista enlazada con nodos en memoria: aquí los "nodos" son registros del archivo, y los "punteros" son índices físicos guardados dentro del propio registro borrado.

Bloque C: Experimento empírico y verificación de representación binaria

Problema 4: Acceso secuencial vs. acceso directo — medición empírica

Enunciado

Generar un archivo binario de 10 000 contactos con `random.seed(42)` (los nombres pueden ser sintéticos: `f"Contacto_{i:05d}"`, los teléfonos `f"{random.randint(1000, 9999)}-{random.randint(1000, 9999)}"`, los emails `f"contacto{i:05d}@fi.uba.ar"`, los ids son los enteros 0 a 9999). Usar el formato sin free list del Problema 2 (`<i32s16s40s'`, 92 bytes), no el extendido del Problema 3.

A continuación, comparar empíricamente dos estrategias para acceder al k -ésimo contacto, midiendo con `time.perf_counter()`:

(a) `acceso_secuencial(ruta, k)` — abre el archivo, lee desde el principio uno tras otro hasta llegar al k -ésimo y devolverlo. No usa `seek` (excepto el implícito al abrir). Es $O(k)$.

(b) `acceso_directo(ruta, k)` — abre el archivo, hace `seek(k * TAM_REGISTRO)` y lee exactamente `TAM_REGISTRO` bytes. Es $O(1)$.

(c) Diseñar un experimento que mida ambos accesos para $k \in \{0, 100, 1000, 5000, 9999\}$ y construir una tabla de resultados con cuatro columnas: índice k , tiempo secuencial (ms), tiempo directo (ms), cociente secuencial/directo. Para cada k , repetir la medición al menos 100 veces y promediar para reducir el ruido del reloj.

(d) Graficar (con `matplotlib` si está disponible, o como tabla ASCII si no) los tiempos en función de k . ¿Cómo varían? ¿Confirma el experimento las complejidades teóricas $O(k)$ y $O(1)$?

(e) Reflexión escrita (5-10 líneas, en comentario multilínea al final del programa): ¿por qué la diferencia es menos dramática de lo que sugiere la teoría asintótica para k pequeños? ¿Qué papel juega el *buffer cache* del sistema operativo (Sección 8.3 del documento teórico) en la medición?

Orientaciones para la resolución

Análisis: El acceso secuencial debe leer y descartar $k-1$ registros antes del que interesa. Para $k = 9999$ esto significa unas 920 KB transferidas innecesariamente. El acceso directo transfiere solo 92 bytes (o más probablemente, un bloque entero de 4 KiB que los contiene).

Diseño: Para que la medición sea limpia, abrir y cerrar el archivo dentro de la función medida (así se simula el caso real donde cada acceso parte de un archivo cerrado). Para reducir el ruido, repetir la operación con la misma k cien veces dentro de un bloque marcado por `t0 = perf_counter()` y `t1 = perf_counter()`, y dividir $(t1 - t0)$ por 100 para obtener el promedio. La caché del SO hará que la segunda lectura del mismo archivo sea más rápida que la primera; esto es realista pero conviene mencionarlo en la reflexión.

Evaluación: El cociente secuencial/directo debe crecer aproximadamente linealmente con k . Para $k = 9999$ se espera un cociente del orden de $100\times$ o más, dependiendo del tamaño del bloque del sistema de archivos y del estado de la caché.

*Conexión con la teoría — Este experimento materializa la afirmación central de la Sección 4.3: el acceso al k -ésimo registro es $O(1)$ en archivos binarios de longitud fija. La discrepancia entre la teoría asintótica y la medición práctica está mediada por el *buffer cache* (Sección 8.3) y por el tamaño de bloque del sistema de archivos (Sección 8.1): la primera lectura de un bloque cuesta una transferencia desde el medio físico, las siguientes dentro del mismo bloque son prácticamente gratis.*

Problema 5: Verificación de representación binaria — Semana 2 revisitada

Enunciado

En la Semana 2 estudiamos cómo se representan los números en binario: enteros con signo en complemento a dos, números reales en IEEE 754. En aquel momento la afirmación quedó como abstracción — no teníamos manera de "ver" los bytes de un float. Ahora con `struct` podemos verificarlo empíricamente. Diseñar un programa que valide:

- (a) Para los enteros 1, -1, 127, -128, 32767, -32768: empaquetarlos con '`<b`' (int8) cuando entren, con '`<h`' (int16) cuando entren, y con '`<i`' (int32). Mostrar los bytes en hexadecimal para cada caso. Verificar manualmente que -1 en complemento a dos a 8 bits es `0xFF`, a 16 bits es `0xFFFF`, a 32 bits es `0xFFFFFFFF`. Verificar que -128 en int8 es `0x80`, y que -32768 en int16 es `0x8000`.
- (b) Para los enteros 128, 256, 32768: intentar empaquetarlos con '`<b`', '`<h`'. Algunos darán error de overflow (`struct.error`). Registrar cuáles dan error y justificar con el rango de cada tipo. ¿Por qué 128 no entra en '`<b`' pero sí entra en '`<B`' (uint8)?
- (c) Para los flotantes 0.0, 1.0, -1.0, 0.5, 1.5, 3.14: empaquetarlos con '`<f`' (float32, IEEE 754 single precision) y mostrar los bytes en hexadecimal. Verificar manualmente al menos uno: por ejemplo, 1.0 en IEEE 754 single es `0x3F800000` (en little-endian queda `00 00 80 3F`). Consultar la sección de IEEE 754 de la Semana 2 para reconocer los campos de signo, exponente y mantisa en los bits.
- (d) Empaquetar 0.1 con '`<f`' y luego desempaquetarlo. ¿Se recupera exactamente 0.1? Mostrar el resultado con `repr()`. Empaquetar también con '`<d`' (float64) y comparar la pérdida de precisión. Esta es la representación inexacta de 0.1 en binario (que ya vimos en la Semana 2): aquí se materializa.
- (e) Reflexión: ¿por qué los protocolos de red TCP/IP especifican explícitamente network byte order como big-endian? ¿Qué pasaría si dos máquinas con endianness distinto intercambiaran el archivo creado en el Problema 3 sin acordar previamente el formato? Probar invertir el endianness leyendo un archivo del Problema 3 con '`>i32s16s40s`' (big-endian) y observar el desastre que produce en los enteros.

Orientaciones para la resolución

Análisis: El complemento a dos para -1 a n bits es $2^n - 1$, que en hexadecimal es $n/4$ dígitos F. El IEEE 754 single divide los 32 bits en 1 de signo + 8 de exponente + 23 de mantisa: para 1.0 el signo es 0, el exponente es `0x7F` (bias 127), la mantisa es 0, lo que da `0 01111111 000000000000000000000000` = `0x3F800000`.

Diseño: Usar la función `bin(int.from_bytes(b, 'little'))` para convertir bytes a su representación binaria explícita (útil para reconocer los campos del float). Para los casos de overflow, encerrar el `struct.pack` en un `try/except struct.error` y reportar el valor que falló y el motivo.

Evaluación: Verificar contra calculadoras online de IEEE 754 (por ejemplo, www.h-schmidt.net/FloatConverter/IEEE754.html) los casos del punto (c). La discrepancia esperada en

(d) muestra que 0.1 no tiene representación finita en binario, exactamente como 1/3 no tiene representación decimal finita.

Conexión con la Semana 2 — Este problema cierra el ciclo abierto en la Semana 2: la representación binaria de números no es solo una abstracción para entender los rangos y la precisión, sino la disposición concreta de bytes que escribimos al disco y que enviamos por red. El módulo struct es la ventana que nos permite "ver" esa representación.

Análisis crítico de implementaciones generadas por IA — Si en algún momento se utiliza una IA generativa para resolver alguno de los problemas (especialmente los del Bloque B), revisar la propuesta antes de aceptarla:

1. ¿Maneja correctamente el endianness? Una solución que omita el prefijo ('i' sin '<') parecerá funcionar en la PC del estudiante pero fallará en otra arquitectura. 2. ¿Hardcodea el tamaño del registro como 92 o 97? La buena práctica es derivarlo con struct.calcsize. Si la IA escribe archivo.read(97) en lugar de archivo.read(TAM_REGISTRO), el código se romperá silenciosamente cuando alguien cambie el formato. 3. ¿Usa 'r+b' correctamente para actualización? Un error común es abrir en 'wb' (que trunca el archivo) cuando se quiere actualizar in situ; otro error es olvidar el seek antes de escribir. Verificar siempre con un volcado hexadecimal después de la operación. 4. ¿La free list está implementada correctamente? Pedirle a la IA que muestre la traza de tres altas seguidas de dos bajas y una nueva alta. Si la traza no coincide con la predicción del estudiante hecha a mano, alguna de las dos está mal — encontrar cuál.

Desafíos extracurriculares grupales

Esta semana, los desafíos extracurriculares se proponen para **trabajo grupal** (grupos de 2 o 3 estudiantes), distribuyéndose en aproximadamente **6 horas** durante la semana. Cada grupo elige **uno** de los cuatro desafíos siguientes y entrega: código completo con docstrings y comentarios, archivo de prueba binario producido por el programa, capturas de pantalla del volcado hexadecimal correspondiente, y un informe breve (1-2 páginas) que justifique las decisiones de diseño y compare la implementación lograda con la free list de la Sección 5 del documento teórico.

Desafío 1: ABM con bitmap auxiliar de espacio libre

Enunciado

Reimplementar el ABM de la agenda usando **un archivo auxiliar de bitmap** (Sección 5.5 del documento teórico) en lugar de la free list encadenada. El archivo de datos agenda.dat y el archivo de bitmap agenda.bmp se gestionan en paralelo: cada bit del bitmap indica si el registro homólogo está libre (1) o ocupado (0).

Consignas:

- (a) Mantener el byte activo dentro del registro de datos para que los recorridos secuenciales no necesiten leer el bitmap. Eliminar el campo `siguiente_libre` (el registro queda en 93 bytes: `'<Bi32s16s40s'`).
- (b) Implementar las operaciones `bit_libre(bitmap, k)`, `marcar_libre(bitmap, k)` y `marcar_ocupado(bitmap, k)` con aritmética de bits (`>>`, `<<`, `|`, `&`, `~`).
- (c) Implementar `buscar_slot_libre(bitmap)` con la mejora del cursor del último libre encontrado: la búsqueda parte desde la última posición devuelta y da la vuelta al final, en lugar de empezar siempre desde 0.
- (d) Implementar las cuatro operaciones del ABM: alta, baja, modificacion, listar_activos. Mantener consistencia entre ambos archivos: si el programa se interrumpe abruptamente entre la escritura del registro y la actualización del bit, ¿cómo se podría recuperar la consistencia al reabrir? (Pista: una pasada inicial que verifique la coherencia entre los bits del bitmap y los flags activo de los registros).
- (e) Implementar `contar_libres()` en $O(n/64)$ usando `bin(byte).count('1')` para contar bits encendidos byte por byte. Comparar con la versión equivalente sobre la free list (que necesita recorrerla nodo por nodo).
- (f) Comparación experimental: ejecutar la misma secuencia de operaciones (1000 altas, 500 bajas aleatorias, 500 altas más) sobre la implementación con free list (Problema 3) y sobre la implementación con bitmap. Medir tiempos totales y tamaño final de los archivos. Discutir los resultados.

Entregable adicional

Un informe breve que responda: ¿en qué condiciones es preferible cada implementación? ¿Por qué los sistemas de archivos reales (ext4, NTFS, APFS) eligen mayoritariamente bitmaps?

Desafío 2: Catálogo en bloques de longitud fija con índice persistente**Enunciado**

Implementar un catálogo de productos con descripciones de longitud variable organizado en **bloques de tamaño fijo** de 4096 bytes (Sección 6.2 del documento teórico, con compactación inmediata en cada baja). Cada producto tiene tres campos: `id` (int32, único), `nombre` (cadena variable de hasta 64 caracteres) y `descripcion` (cadena variable de hasta 1000 caracteres). Los registros se identifican externamente por su `id` del dominio (no por su posición física dentro del bloque, que es inestable porque la compactación los desplaza tras cada baja).

Para resolver eficientemente la pregunta *"¿en qué bloque está el producto con `id = X`?"* sin recorrer todo el archivo, el catálogo se acompaña de un **índice persistente**: una lista de tuplas `[(id, n_bloque), ...]` ordenada por `id`, serializada como JSON al cerrar el catálogo y recargada al abrirlo. La búsqueda en el índice es $O(\log n)$ con búsqueda binaria (Semana 5), la inserción mantiene el orden con desplazamiento $O(n)$. El índice es liviano —dos enteros por producto— y se carga entero a memoria, mientras que el archivo de datos puede ser arbitrariamente grande y se accede por bloques. Esta arquitectura híbrida —**binario eficiente para los datos + JSON portable**

para los metadatos— es la que usan motores de bases de datos como SQLite y muchos formatos de archivo modernos (Parquet, HDF5).

Consignas:

(a) Definir la estructura física del catálogo: tres archivos en paralelo, gestionados consistentemente por el conjunto de operaciones del catálogo.

Archivo	Contenido	Formato
catalogo.dat	Bloques de 4096 bytes con registros de longitud variable	Binario, organización interna a definir en (b)
catalogo.idx.json	Lista ordenada [[id, n_bloque], ...]	JSON (texto, portable)
catalogo.fsm	uint16 por bloque con su espacio libre actual	Binario '<H' por entrada

(b) **Decisión de diseño grupal:** cada equipo elige **una de las tres alternativas siguientes** para organizar los registros dentro de cada bloque, y **fundamenta la elección** en el informe (un párrafo) discutiendo simplicidad, eficiencia de búsqueda intra-bloque, costo de las operaciones de baja y modificación, y conexión visual con el diagrama de la Sección 6.2 del documento teórico.

Alternativa 1 — Concatenación con prefijo de longitud. Los registros se almacenan uno tras otro al inicio del bloque, cada uno precedido por su longitud total (uint16). La búsqueda intra-bloque es lineal: leer el prefijo, comparar el id del registro, y si no coincide saltar al próximo prefijo sumando 1en bytes. La cabecera del bloque tiene 4 bytes: n_registros (uint16) y fin_datos (uint16, primer byte libre tras el último registro). Es la organización más simple, alineada con lo visto en Sección 6.1; sin directorio ni complejidad bidireccional.

Alternativa 2 — Slotted pages clásicas con directorio compactado. Geometría completa de la Sección 6.2: cabecera (n_registros, fin_datos, ini_slots), zona de datos creciendo desde el inicio, directorio de ranuras (offset, 1en) creciendo desde el final. **Sin slots muertos:** en cada baja se elimina la entrada del directorio y se compactan los registros vivos. La búsqueda intra-bloque consulta el directorio (acceso aleatorio dentro del bloque) en lugar de recorrer secuencialmente. Mantiene el contacto visual con el SVG del documento teórico.

Alternativa 3 — Híbrido: directorio liviano de offsets sin manejo de huecos. Cabecera con n_registros + fin_datos + ini_offsets, zona de datos al inicio, y al final una lista compacta de offsets (uint16) sin longitudes (la longitud se obtiene leyendo el prefijo del registro al llegar a su offset). La búsqueda intra-bloque consulta el offset, salta a esa posición y compara el id. Más simple que la alternativa 2, más rápido para acceso aleatorio que la alternativa 1, pero pierde la posibilidad de saltar el contenido de un registro sin leerlo.

Pauta de procesamiento de bloques — tres niveles de representación:

Antes de implementar las operaciones del catálogo, conviene fijar el modelo mental que sostendrá toda la lógica intra-bloque. Cuando ejecutamos `archivo.seek(n_bloque * TAM_BLOQUE)` seguido de `archivo.read(TAM_BLOQUE)`, lo que llega a memoria es un objeto `bytes` de Python de exactamente 4096 bytes —una secuencia plana e inmutable—. El sistema operativo transfiere el bloque entero del disco a la RAM de una sola vez, así que **leer un bloque cuesta lo mismo independientemente de cuánto de él esté ocupado** (Sección 8.1 del documento teórico). Una vez en memoria, ese objeto `bytes` puede sectorizarse con slicing (`bloque[4:8]` da cuatro bytes) y desempaquetarse por regiones con `struct.unpack_from(formato, bloque, offset)`, que opera directamente sobre la secuencia original sin copiarla.

Para *modificar* el bloque necesitamos una secuencia mutable: la solución estándar es convertirlo a `bytearray` con `buffer = bytearray(bloque)`. El `bytearray` acepta tanto asignación por índice (`buffer[0] = 42`) como asignación por slice de longitud variable (`buffer[10:20] = nuevo_contenido`, donde `nuevo_contenido` puede ser de longitud distinta a la del slice; el `bytearray` crece o se encoge automáticamente). Esta operación es justamente la que sostiene la compactación: eliminar un registro y desplazar los siguientes a la izquierda es un slice assignment. Al terminar la modificación, escribimos de vuelta con `seek + write(bytes(buffer))`, asegurando que el `bytearray` vuelva a tener exactamente 4096 bytes (rellenando con ceros la cola si quedó más corto).

Sobre estos bytes operan **tres niveles de representación que coexisten durante una operación**: (1) el bloque crudo como `bytes` o `bytearray`, fiel al disco, de tamaño siempre 4096; (2) los registros desempaquetados como tuplas (`id, nombre, descripcion`) que son cómodas para comparar por igualdad y para imprimir, pero que son copias temporales sin conexión con los bytes originales; (3) la lista Python de tuplas con todos los registros vivos del bloque, que se construye desempaquetándolos uno por uno en un `for`. La operación típica que toca varios registros del mismo bloque (la compactación al dar una baja es el caso paradigmático) se vuelve naturalmente legible cuando se piensa en términos de esta lista intermedia: filtrar el registro a borrar es un `for` que va acumulando los que sobreviven; reconstruir el bloque consiste en empaquetar cada registro de la lista filtrada en un `bytearray` nuevo de 4096 bytes, escribir antes la cabecera actualizada, y volcar al disco.

Este patrón —**leer bloque → desempaquetar a lista → operar sobre la lista → reempaquetar bloque → escribir bloque**— es exactamente el mismo que se usó en la Semana 6 con archivos de texto (leer archivo → parsear a lista de tuplas → operar → reformatear → reescribir el archivo). La diferencia clave es que ahora la unidad de trabajo es un bloque de 4 KiB en lugar del archivo completo, lo que permite procesar catálogos arbitrariamente grandes sin cargar todo a memoria. La generalización vale para todas las operaciones de modificación del catálogo: cambia el contenido específico de cada paso (qué se filtra, qué se agrega, qué se reordena), pero la estructura del bucle y la convivencia de los tres niveles de representación se mantiene.

Una observación importante sobre el flujo completo de `borrar_producto` ilustra cómo se articulan las distintas estructuras en RAM con los archivos en disco: primero se localiza el bloque consultando la **lista índice** ya cargada en memoria con búsqueda binaria —una operación $O(\log n)$ sobre la estructura Python sin tocar el disco—; luego se trae el bloque a RAM con un `seek + read` sobre `catalogo.dat`; el bloque se procesa en los tres niveles para producir la versión compactada;

se reescribe con un `seek + write` en su posición original; se actualiza el **FSM** con una escritura puntual de 2 bytes en la posición `n_bloque` de `catalogo.fsm`; y finalmente se elimina la entrada (`id`, `n_bloque`) de la **lista índice en RAM**. Este último paso *no* desencadena ninguna escritura del archivo `catalogo.idx.json` —el índice se persiste solo al cerrar el catálogo en `cerrar_catalogo`, evitando reescribir el JSON en cada operación—. La elegancia del diseño está en que **los tres niveles de representación son transitorios salvo el primero**: las tuplas y las listas viven solo durante la operación, los archivos en disco son la única verdad persistente, y la gestión explícita de las transiciones entre niveles es lo que hace que el sistema sea robusto.

(c) Implementar `abrir_catalogo(ruta_base)` que reciba el prefijo de los archivos (por ejemplo `'catalogo'`), cargue el índice desde `ruta_base + '.idx.json'` con `json.load`, abra `ruta_base + '.dat'` en modo `'r+b'` y `ruta_base + '.fsm'` también en modo `'r+b'`, y devuelva una tupla con los handles abiertos y la lista índice cargada. Si los archivos no existen aún, los crea (índice = lista vacía, FSM y dat vacíos).

(d) Implementar `cerrar_catalogo(estado)` que vuelque la lista índice de vuelta a `'.idx.json'` con `json.dump` (con `indent=2` para legibilidad humana del índice), cierre los archivos abiertos, y se asegure de hacer `flush` antes del cierre para que los datos lleguen al disco.

(e) Implementar `agregar_producto(estado, id, nombre, descripcion)`. Pasos: verificar con búsqueda binaria sobre el índice que el `id` no exista (si existe, levantar `ValueError`); calcular el tamaño total que ocupará el registro serializado en el bloque (varía según la alternativa elegida en (b)); buscar en el FSM el primer bloque cuyo espacio libre alcance, o crear un bloque nuevo al final del archivo si no hay; abrir el bloque, agregar el registro al inicio (alternativa 1) o donde corresponda según el directorio (alternativas 2 y 3); reescribir el bloque completo, actualizar el FSM con el nuevo espacio libre, e insertar (`id`, `n_bloque`) en la posición ordenada del índice (búsqueda binaria + desplazamiento).

(f) Implementar `leer_producto(estado, id)`. Pasos: búsqueda binaria sobre el índice para localizar el `n_bloque` del producto (si no aparece, devolver `None`); leer el bloque entero con `seek(n_bloque * TAM_BLOQUE)` y `read(TAM_BLOQUE)`; recorrer los registros del bloque según la organización elegida en (b) hasta encontrar el que tenga el `id` buscado; devolver la tupla (`id`, `nombre`, `descripcion`).

(g) Implementar `borrar_producto(estado, id)` con **compactación inmediata**. Pasos: localizar el bloque vía índice; leer el bloque entero; identificar el registro con el `id` solicitado; reconstruir el bloque sin ese registro, desplazando los registros vivos para cerrar el hueco (la mecánica exacta del desplazamiento depende de la alternativa elegida en (b)); reescribir el bloque completo en disco; actualizar el FSM con el nuevo espacio libre; eliminar la entrada (`id`, `n_bloque`) del índice. Justificar en el informe por qué en este contexto pedagógico no necesitamos baja lógica diferida (los argumentos —identidad estable de identificadores externos, concurrencia, rollback transaccional— que justifican el diseño de PostgreSQL no aplican a un programa monolítico monousuario sin índices secundarios).

(h) Implementar `recorrer_catalogo(estado)` como una función que devuelva todos los productos en su **orden físico real de almacenamiento** —recorriendo `catalogo.dat` bloque por bloque, y dentro de cada bloque registro por registro—, **sin usar el índice**. La salida puede ser una lista de tuplas (`n_bloque`, `id`, `nombre`, `descripcion`). Esta función permite inspeccionar visualmente cómo se distribuyen los registros tras una secuencia de altas y

bajas, y verificar que el orden físico difiere del orden lógico que arroja recorrer el índice (ordenado por id).

(i) Prueba integradora: cargar un catálogo de 100 productos con `random.seed(42)` (ids del 1 al 100, nombres y descripciones de longitudes dispares generados pseudoaleatoriamente). Cerrar y reabrir el catálogo verificando que el índice se persistió correctamente. Borrar 30 productos al azar (anotar cuáles). Verificar con `recorrer_catálogo` que efectivamente quedan 70 productos en el archivo y que están agrupados sin huecos dentro de cada bloque (consecuencia de la compactación inmediata). Para cinco ids específicos (algunos borrados, algunos vivos), verificar que `leer_producto` devuelve el resultado correcto en cada caso.

Entregable adicional

El informe del grupo debe incluir:

1. Justificación de la **alternativa de organización de bloque elegida** en (b), con un análisis comparativo breve de las tres alternativas en términos de simplicidad de implementación, costo de búsqueda intra-bloque, costo de baja con compactación, y conexión con los conceptos del documento teórico.
2. Un **diagrama** (dibujado a mano y escaneado, o construido digitalmente) de un bloque del catálogo después de la secuencia de pruebas, mostrando la cabecera, los registros vivos según la alternativa elegida, y el espacio libre. Comparar con el SVG de la Sección 6.2 del documento teórico cuando corresponda.
3. Una **discusión** breve sobre el diseño del índice: ¿por qué se eligió una lista de tuplas ordenada en lugar de un diccionario? (pista: la planificación del curso reserva los diccionarios para la Semana 9, pero además, ¿qué otras razones hay para preferir una lista ordenada cuando se quiere persistir el índice como JSON portable y legible?). ¿Qué pasaría si el catálogo creciera a un millón de productos —el índice seguiría siendo viable como lista, o convendría reorganizarlo (por ejemplo, en un árbol B+, como hacen los motores reales)?
4. Una **inspección hexadecimal** de los primeros dos bloques del archivo `catalogo.dat` después de la prueba (con `xxd | head -50` o equivalente), señalando byte por byte la cabecera del bloque y al menos un registro completo.

Desafío 3: Log append-only con índice y reconstrucción

Enunciado

Implementar un sistema de persistencia de eventos con la arquitectura **log append-only** (Sección 6.1 del documento teórico). Cada evento tiene: `timestamp` (int64, segundos desde epoch), `tipo` del evento (cadena de hasta 32 bytes) y `payload` (cadena variable de hasta 4096 bytes).

Consignas:

- (a) Formato del registro en el log: `'<q' + len_tipo (1 byte) + tipo + len_payload (2 bytes) + payload`. La longitud total del registro es `8 + 1 + len_tipo + 2 + len_payload` bytes, sin padding.
- (b) Implementar `agregar_evento(timestamp, tipo, payload)` que serialice el evento, abra el log en modo `'ab'` (append binario), escriba el registro, y simultáneamente agregue al archivo

índice `eventos.idx` el offset (int64) donde quedó el inicio del nuevo evento. Devolver el `k` del nuevo evento.

(c) Implementar `leer_evento(k)` que use el índice para obtener el offset del evento `k`-ésimo y devuelva la tupla (`timestamp`, `tipo`, `payload`). Es $O(1)$ gracias al índice.

(d) Implementar `recorrer_eventos()` como un generador (`yield`) que recorra el log secuencialmente leyendo los prefijos de longitud, sin usar el índice. Es la forma de recorrido que se usa al reconstruir agregaciones (suma de eventos por tipo, conteo, máximos, etc.).

(e) Implementar `reconstruir_indice()` que tome un log existente y regenere desde cero el archivo de índice recorriéndolo secuencialmente. Esta operación es esencial para recuperarse de un índice corrupto o perdido. Verificar que después de la reconstrucción, las consultas con `leer_evento(k)` producen los mismos resultados que antes.

(f) Generar un log de prueba con 5000 eventos de tipos variados ('login', 'logout', 'purchase', 'error') con payloads de longitudes diversas. Implementar una función `agregar_por_tipo()` que recorra el log con el generador y devuelva un diccionario {`tipo`: `cantidad`} (para esta semana, simular el diccionario con una lista de tuplas; los diccionarios de Python se introducen en la Semana 9).

Entregable adicional

Un experimento que mida: tiempo de lectura del último evento usando el índice vs. tiempo de recorrido secuencial completo. Mostrar que el primero es esencialmente constante mientras que el segundo crece linealmente con el tamaño del log. Comparar con el experimento del Problema 4.

Desafío 4: Migración entre formatos — struct ↔ JSON Lines

Enunciado

Implementar herramientas de migración bidireccional entre el formato binario '`<i32s16s40s`' del Problema 2 y el formato textual JSON Lines (Sección 7.3 del documento teórico). Esto permite, por ejemplo, exportar la agenda binaria a un formato legible por humanos y portable a otros lenguajes, y reimportarla a binario para procesamiento eficiente.

Consignas:

(a) Implementar `binario_a_jsonl(ruta_binaria, ruta_jsonl)` que abra el archivo binario, recorra todos los registros activos y escriba cada uno como una línea JSON en el archivo de salida con el formato {"id": ..., "nombre": "...", "telefono": "...", "email": "...". Las cadenas se desempaquetan con `.rstrip(b'\x00').decode('utf-8')` para remover el padding.

(b) Implementar `jsonl_a_binario(ruta_jsonl, ruta_binaria)` que lea el archivo JSON Lines línea por línea (sin cargar todo a memoria) y produzca un archivo binario equivalente. Las cadenas más largas que el campo se truncan; las más cortas se rellenan automáticamente al empaquetar.

(c) Implementar `procesar_streaming(ruta_jsonl, dominio)`: cuenta cuántos contactos del archivo JSON Lines tienen su email en el dominio dado, recorriendo línea por línea sin cargar el

archivo. Probar con un archivo grande para confirmar que el uso de memoria es constante (independiente del tamaño del archivo).

(d) Comparación de tamaños: para una agenda de 10 000 contactos, comparar el tamaño en disco de las tres representaciones: binario con struct (920 KB), JSON Lines (≈ 1.2 -1.5 MB), JSON plano con indentación de 2 espacios (≈ 2 MB o más). Discutir el trade-off entre tamaño y portabilidad.

(e) Reflexión integradora: ¿en qué momento del ciclo de vida de los datos conviene cada formato? Sugerencia: pensar en los siguientes escenarios y proponer el formato apropiado para cada uno: persistencia operativa de la aplicación, exportación para consumo por sistemas externos, backup archivado, intercambio con un programa Java que también consume la agenda, log de auditoría que se escribe constantemente y rara vez se lee.

Entregable adicional

Una propuesta de **arquitectura de capas** para la agenda real: en qué formato vive durante la operación (escrituras frecuentes, búsquedas por id), en qué formato se exporta a otros sistemas, en qué formato se respalda. Justificar cada elección con criterios de los formatos vistos en la Sección 7 del documento teórico.

Distribución sugerida del tiempo (9 horas)

Actividad	Problemas / Tareas	Tiempo	Contexto
Clase práctica (3 h)	Problema 1	~30 min	Familiarización con struct, endianness, calcsz
	Problema 2	~45 min	Diseño del registro y operaciones básicas
	Problema 3 (parcial)	~1 h 15 min	ABM con free list + verificación hexadecimal en clase
	Problema 4 (planteo)	~30 min	Diseño del experimento empírico, dejar la corrida para extracursado
Extracursado individual (3 h)	Problema 3 (finalización)	~30 min	Completar listar_activos y la traza hexadecimal del caso de Margaret
	Problema 4 (medición)	~1 h 15 min	Generar archivo de 10 000 registros, ejecutar el experimento, analizar

	Problema 5	~1 h	Verificación de complemento a dos e IEEE 754
	Lectura: documentación oficial del módulo struct	~15 min	docs.python.org/3/library/struct.html
Extracursado grupal (6 h)	Selección y planificación del desafío	~30 min	Equipos de 2-3 estudiantes eligen uno de los cuatro desafíos
	Implementación del desafío elegido	~4 h 30 min	Codificación, pruebas, volcado hexadecimal de verificación
	Redacción del informe	~1 h	Justificación de decisiones, comparación con la free list

Criterio de distribución — Los Problemas 1, 2 y 3 forman el núcleo formativo y se inician en clase con asistencia presencial; el Problema 3 se completa en extracursado porque la traza manual requiere tiempo individual. El Problema 4 se planifica en clase y se ejecuta en extracursado porque la generación del archivo de 10 000 registros y la corrida con cien repeticiones por k toman varios minutos sin valor pedagógico adicional en clase. El Problema 5 es individual por su carácter de verificación conceptual. Los desafíos extracurriculares grupales (6 horas) se ofrecen para profundización con autonomía de equipo, eligiendo uno de cuatro temas que cubren las áreas conceptualmente introducidas pero no implementadas en clase.

Notas finales

Estos cinco problemas más cuatro desafíos grupales cubren los contenidos prácticos de la **Semana 8** del curso. Los Problemas 1 y 2 construyen las herramientas básicas (struct, registros fijos, acceso directo). El Problema 3 implementa el patrón ABM completo con free list encadenada y verificación visual mediante volcado hexadecimal. El Problema 4 valida empíricamente la ganancia teórica del acceso directo sobre el secuencial. El Problema 5 cierra el ciclo con la Semana 2 verificando que la representación binaria de enteros y reales se materializa exactamente como predijo la teoría. Los cuatro desafíos extracurriculares grupales profundizan las áreas que la teoría introdujo conceptualmente sin implementar (bitmap, slotted pages, log append-only, JSON Lines), preparando el terreno para los temas avanzados de las semanas siguientes.

Criterios de evaluación formativa

Además de los criterios de semanas anteriores (metodología de Pólya, modelo de programa, *literate programming*, contratos de funciones, manejo de efectos secundarios), esta semana se evaluarán especialmente:

Constantes derivadas como fuente única de verdad: `TAM_REGISTRO` = `struct.calcsize(FORMATO)` y nunca un literal 92 o 97 hardcodeado en el código.

Manejo explícito de endianness: el prefijo < debe aparecer en todos los formatos, justificando su elección frente a la portabilidad entre arquitecturas.

Documentación de efectos secundarios sobre archivos: las funciones que crean, modifican o borran datos en el archivo lo declaran explícitamente en la docstring.

Verificación con volcado hexadecimal: al menos en el Problema 3, presentar el volcado hexadecimal del archivo después de la secuencia y señalar byte por byte la cabecera, los registros activos, los registros borrados y los enlaces de la free list.

Análisis empírico riguroso: en el Problema 4, las mediciones se promedian sobre múltiples corridas; los gráficos o tablas tienen ejes correctamente rotulados; las conclusiones se derivan de los datos y no de expectativas previas.

Trabajo grupal en los desafíos: el informe identifica claramente la contribución de cada integrante; el código pasa por revisión cruzada antes de la entrega.

Para profundizar

(1) En el Problema 1, investigar la diferencia entre el prefijo '@' (alineación nativa con padding) y '<' o '>' (sin padding): ¿por qué `struct.calcsize('@hi')` puede dar 8 en lugar de 6? ¿Qué tiene que ver con la alineación de memoria? (2) En el Problema 3, modificar el ABM para que la cabecera incluya un campo adicional `version` (`uint8`) que se incrementa cada vez que se modifica el archivo: ¿cómo afecta esto al diseño? (3) En el Problema 4, ejecutar el experimento con un archivo de 1 000 000 de registros (≈ 90 MB) y observar cómo cambia el cociente secuencial/directo: para k cercano a un millón, el secuencial debería tardar segundos mientras que el directo sigue siendo instantáneo. (4) En el Problema 5, investigar el formato '<e' (`float16`, half precision, IEEE 754 binary16) que algunas GPUs y aceleradores de IA utilizan: ¿cuál es su rango y precisión? ¿Por qué se está volviendo cada vez más popular en el cómputo numérico? (5) Comparar la free list de la Sección 5 con la implementación que hace SQLite del free space en sus archivos `.db`: leer la sección "Free Page List" de la documentación oficial del formato de archivo SQLite.

Guía para el Foro

Formatos de persistencia: cuándo elegir cada uno

El foro de esta semana propone trabajar sobre la taxonomía de la Sección 7 del documento teórico (`struct` + registros fijos / JSON Lines / JSON plano / pickle) aplicándola a casos reales:

Consigna: Cada estudiante presenta **un caso de uso concreto** de su área de interés (puede ser ficticio, basado en proyectos personales o tomado de su experiencia laboral) que requiera persistencia de datos. Para ese caso, debe:

(a) Identificar si la situación es de tipo "objeto completo" (Pregunta A del documento) o "colección de registros" (Pregunta B), y justificar.

(b) Elegir el formato más apropiado entre los cuatro presentados, justificando con criterios técnicos (tamaño esperado, frecuencia de escritura, frecuencia de lectura, necesidad de portabilidad, sensibilidad de los datos).

(c) Anticipar los problemas que aparecerían si se eligiera un formato inadecuado: por ejemplo, ¿qué pasaría si se eligiera JSON plano para un caso que en realidad es de "colección que crece indefinidamente"?

Lectura cruzada: leer al menos dos contribuciones de compañeros y comentar si la elección del formato está bien justificada o si propondrían una alternativa diferente, con argumentos.

Cierre: al final de la semana, el docente sintetizará en un mensaje las elecciones más frecuentes por tipo de caso, identificando patrones recurrentes y errores típicos en la elección.